

Project 5 Technical Summary

For this project, we were able to utilize not only the functions learned in class, but also many others we learned from the internet (github, coding blogs, etc.). Doing so allowed us to create the project in a much more efficient and logical way. Attached below is a brief summary of the logic behind the code in the project, broken down by Module.

Module A — Aggregate Gradation Automator

Purpose: Computes a blended aggregate gradation curve and checks it against NDOT specification control points.

Logic:

- Reads a raw Excel file with no pre-defined headers, parsing sieve sizes, individual aggregate percent-passing values, and blend percentages.
 - Supports runtime blend ratio overrides via a `blend_overrides` dict, with a warning if fractions don't sum to 1.0.
 - Computes the combined gradation as a weighted sum: $\text{combined} = \sum(\text{passing}_i \times \text{blend_pct}_i)$.
 - Compares each sieve result against Spec Min/Max, labeling each point PASS or FAIL.
 - **Output:** A DataFrame with combined gradation and status, plus a log-scale sieve curve plot with the NDOT spec band shaded.
-

Module B — Volumetric Properties Calculator

Purpose: Calculates air voids (V_a) for asphalt specimens using AASHTO T 209 / T 166 methodology.

Logic:

- **Gmm (Maximum Specific Gravity):** Computed from Rice test measurements on Sample 1 only ($G_{mm} = A_{\text{rice}} / (A_{\text{rice}} - B_{\text{rice}})$), then inherited by subsequent samples.
- **Gmb (Bulk Specific Gravity):** Computed per sample from dry, submerged, and SSD masses ($G_{mb} = A_{\text{dry}} / (B_{\text{ssd}} - C_{\text{sub}})$).
- **Air Voids:** $V_a = 100 \times (G_{mm} - G_{mb}) / G_{mm}$.
- **QC threshold:** V_a must fall within 6.5–7.5% to be "Valid for Performance Testing"; anything outside is flagged INVALID.
- **Output:** A bar chart with the target V_a band highlighted and per-bar value annotations.

Module C — IDEAL-CT Cracking Index Analyzer

Purpose: Computes the CT_index from IDEAL Cracking Test (IDEAL-CT) load-displacement curves.

Algorithm:

1. Work of fracture W_f computed via `scipy.integrate.trapezoid`.
2. Fracture energy $G_f = W_f / (t \times D \times 1e-6)$ in J/m^2 .
3. Peak load identified; the post-peak 75% load point found by linear interpolation.
4. Post-peak slope m_{75} computed via `scipy.stats.linregress` on a local window around the 75% crossing.
5. l_{75} = displacement from peak to 75% point.
6. $CT_index = (G_f / (|m_{75}| \times l_{75})) \times (t / D^2)$.

Specimen dimensions are parsed directly from the header. Plots overlay all sample curves with peak and 75% post-peak markers annotated.

Module D — Rutting Analyzer — Hamburg Wheel Tracking Test

Purpose: Analyzes HWTT rut-depth vs. wheel-cycle data to detect moisture damage susceptibility and determine pass/fail outcomes.

Algorithm:

- **SIP Detection (Stripping Inflection Point):** Applies a filter to smooth the rut curve, then computes the first derivative via `np.gradient`. The SIP is the cycle of maximum derivative after skipping the first 20% of data.
- **Failure Cycle:** Linear interpolation to the exact cycle where rut depth crosses the failure threshold (default 12.5 mm).
- **Pass/Fail Verdict:** PASS if 20,000 cycles are survived below threshold; FAIL if threshold is crossed before; INCONCLUSIVE if the test ended early.

Plots display rut depth curves (y-axis inverted), with diamond markers for SIP and X markers for failure points, plus annotated cycle counts.

Additional Notes

- All modules are self-contained functions callable independently with a filepath argument.
- Excel parsing is done header-free throughout, with manual column indexing — making the code tightly coupled to the expected spreadsheet layouts.

- A `__main__` block wires up all four modules sequentially against four named Excel files, saving PNG plots for each.
- Global plot styling is set once via `sns.set_theme` and `plt.rcParams`